Lab11. Running RDS database

Cloud Programming (W04IST–SI4527G)

Wojciech Thomas

Spring 2026

1 Learning objectives

1. Create relational database in the AWS cloud
2. Import database from Cloud9
3. Query database using Python

2 Exercises

2.1 Create MySQL database in the cloud

In this task you configure MySQL database running in "Aurora and RDS" service.

1. Open AWS Console.
2. From Services (top, left corner) choose Database > Aurora and RDS
3. Click Create database button.
4. Select the following options (do not change other options):
    1. "Choose a database creation method": Standard create
    2. "Engine options": MySQL
    3. "Templates": Free tier
    4. "Settings":
        - DB Instance identifier: mydb
        - Master username: admin
        - Credentials management: switch to "Self managed"
        - Master password: lab-password
    5. Expand "Additional configuration" and:
        - Initial database name: lab
        - uncheck "Enable automated backups" option (for faster start),
        - uncheck "Enable encryption"
    6. Click Create database button. You will have to wait few minutes for database to start.
5. In RDS service page select your database. Copy endpoint address from "Connectivity & security" section, and paste it in text editor. Notice the port number.

2.2 Connect to MySQL database and import sample data

In this task you connect to database using command line tool `mysql`, and then you import sample database. You also configure firewall rule, to allow only connections from Cloud9 instance.

1. Open Cloud9 environment.
2. In Bash terminal (at bottom of IDE) type:

   ```
   $ mysql -h <db-endpoint> -P 3306 -u admin -p
   ```

   Replace `<db-endpoint>` with endpoint address copied previously.

   Were you successful? You should not!
3. In order to connect database you have to update Security Group (firewall):
   1. In the RDS service panel select your database.
   2. Select tab "Connectivity & security".
   3. Scroll down to "Security groups rules".
   4. Click security group "default (sg-463463de3)" (number will wary).
   5. On "Security Groups" page select "Inbound rules" tab.
   6. Click "Edit inbound rules" button.
   7. On "Edit inbound rules" page click "Add rule" button.
   8. Select:
      - "Type": All TCP
      - "Source": Custom
      - click magnifier icon and choose "aws-cloud9-..."
   9. Click Save rules button.
4. Try to connect database again.
5. From ePortal, download a `sampledb.sql` file to your computer.
6. In Cloud9 from menu "File" choose "Upload local files...". Upload `sampledb.sql`
7. In the Bash terminal, import sample database:

   ```
   MySQL[(none)]> source ./sampledb.sql
   ```

8. Run SQL query:

   ```
   select * from customers;
   ```

9. Type `exit` to exit MySQL shell.

2.3 Using database from Python

In this step you run simple Python script that connects to MySQL database, and lists the content of the Customers table.

1. In order to use `mysql` module in Python, you have to install its dependencies in Bash terminal:

```
python3 -m pip install --user mysql-connector-python
```

2. Create a new Python file `app.py` and paste the following content inside:

```
import mysql.connector

mydb = mysql.connector.connect(
    host="<your-db-endpoint>",
    user="admin",
    password="lab-password",
    database="sampledb"
)

# List all customers
mycursor = mydb.cursor()
mycursor.execute("SELECT * FROM customers")
myresult = mycursor.fetchall()

for x in myresult:
    print(x)
```

   Replace all `<your-db-endpoint>` string with the correct value.

3. Run the file and watch the result.

2.4 Use Python code to modify the database

In this step you run Python code that adds a new empoyee in the Employees table of MySQL database.

1. Change the Python code to display the content of the `employees` table. When you run your code, you should see the list of company employees.

2. In the **app.py**, before the line `mycursor = mydb...` paste the following code:

```
mycursor = mydb.cursor()
mycursor.execute("INSERT INTO `employees`"
  "(`employeeNumber`, `lastName`, `firstName`, `extension`, "
  " `email`,`officeCode`,`reportsTo`,`jobTitle`)"
  "VALUES "
  "('2000', 'Jane', 'Doe', 'xd101', "
  " 'jdoe@mail.com', '1', '1102', 'Student')")
mydb.commit()
```

3. Run your Python code, and verify if your new employee is visible in the database.

   Tip: If you want to run again this code, you must update `employeeNumber`, as it must be unique.

2.5 Use PHPMyAdmin to connect database

For many years, the PHPMyAdmin was one of the most popular web applications, allowing to remotely manage relational databases, such as MySQL. In this task you will install PHPMyAdmin on Amazon Linux 2023, and connect to the MySQL database running in RDS service.

1. Create EC2 instance with Amazon Linux 2023, with the following settings:
    1. Name: phpmyadmin
    2. Keypair: vockey
    3. Firewall: note the security group name, and select "Allow HTTP traffic from internet"
    4. Advanced details > User data: paste the whole script (remember to merge phpMyAdmin URL into one line!):

```
#!/bin/bash
dnf update
dnf install -y httpd wget php-fpm php-mysqli php-json php php-devel
mkdir -p /var/www/html/phpmyadmin
wget https://files.phpmyadmin.net/phpMyAdmin/5.2.3/phpMyAdmin-5.2.3-all-languages.tar.gz
tar -xvf phpMyAdmin-5.2.3-all-languages.tar.gz \
   -C /var/www/html/phpmyadmin --strip-components 1
cd /var/www/html/phpmyadmin
cp config.sample.inc.php config.inc.php
systemctl start httpd
systemctl enable httpd
```

2. Open "Aurora and RDS" service, select your database.
3. In "Connectivity & security" tab, scroll to "Security group rules". Click any of inbound rules. Click the ID of security group. Select "Inbound rules" tab and click "Edit inbound rules".
4. Click "Add rule". Select Type: MYSQL/Aurora. As Source, select the name of EC2 instance security group, created in this task. Click "Save rules".
5. Find your EC2 instance, and its public IPv4 DNS address.
6. Open browser, and paste the following address: `http://<ec2-pub-dns>/` . You should see the default page of the Apache web server: `It works!`.
7. In EC2 service, select "Instances", then select your EC2 instance. Click the "Connect" button above the list of instances. Click "Connect".
8. In Bash terminal configure the configuration file:

```
$ cd /var/www/html/phpmyadmin
$ sudo nano config.inc.php
```

   In the Nano editor, in the line: `$cfg['Servers'][$i]['host'] = 'localhost';` replace `localhost` with the name of your database endpoint.

9. Restart Apache to reload application:

```
$ sudo systemctl restart httpd
```

10. Open the web page `http://<ec2-pub-DNS>/phpmyadmin/`. Type `admin` and `lab-password`, to log in to your database.

    On the left, expand `sampledb` database, and then expand `customers` table. Compare the result to the result displayed by the Python code.